

Estructuras de Datos y Algoritmos

Tema 7: Pilas

Dr. Jorge Armando Ramírez Díaz

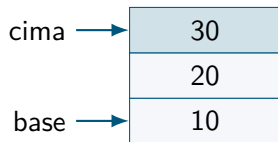
Universidad Nacional Autónoma de México
ENES Unidad Juriquilla
Licenciatura en Matemáticas para el Desarrollo

Una pila restringe las operaciones de una lista a un solo extremo: la **cima**. Este comportamiento se llama **LIFO** (*Last In, First Out*): **último en entrar, primero en salir**.

- El **TDA pila** establece este comportamiento.
- La **representación** puede usar un arreglo dinámico o nodos enlazados.
- Una `list` de Python no es una pila por sí misma: podemos usarla como pila mediante una interfaz restringida.

Definición: TDA pila

Una pila es una secuencia finita de elementos en la que la inserción y la remoción se realizan en un extremo llamado cima, y solo puede consultarse el elemento situado en ese extremo. Sigue el principio LIFO (*last in, first out*), lo que significa que el último elemento agregado es el primero que se retira.



apilar
desapilar

Estado de base a cima:

$$S = (10, 20, 30).$$

La cima contiene 30.

Pila vacía: $S = ()$.

No tiene un elemento en la cima.

Operación	Efecto	Devuelve
<code>apilar(x)</code>	Agrega x en la cima.	None
<code>desapilar()</code>	Retira el elemento de la cima.	Dato retirado
<code>cima()</code>	Consulta la cima sin modificar la pila.	Dato consultado
<code>esta_vacia()</code>	Consulta si no hay elementos.	True o False
<code>longitud()</code>	Consulta cuántos elementos hay.	Entero no negativo

No ofrecemos búsqueda, acceso por índice ni eliminación de un elemento interior.

Especificación

Sea $S = (s_0, \dots, s_{m-1})$, escrito de **base a cima**. Para una pila no vacía, la cima es s_{m-1} .

Operación	Estado posterior S'	Resultado
apilar(x)	$(s_0, \dots, s_{m-1}, x)$	None
desapilar()	$(s_0, \dots, s_{m-2})$	s_{m-1}
cima()	S	s_{m-1}

Si $m = 1$, desapilar deja la secuencia vacía. Si $m = 0$, apilar x deja la secuencia (x) .

- En una pila vacía, cima() y desapilar() producen IndexError y no modifican el estado.
- Se permiten valores repetidos y elementos como None.

Pila con list

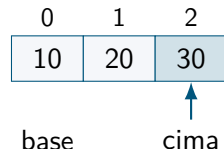
```
class Pila:
    def __init__(self, lista_datos=[]):
        self._datos = lista_datos

    def esta_vacia(self):
        return len(self._datos) == 0

    def longitud(self):
        return len(self._datos)
```

`_datos` es un atributo interno por convención. Python no impide acceder a él, pero no forma parte de la interfaz del TDA.

Un estado posible



`_datos = [10, 20, 30]`

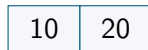
La cima ocupa el último índice. La interfaz no ofrece modificaciones por índice.

Apilar

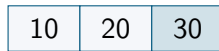
```
def apilar(self, x):  
    self._datos.append(x)
```

```
p = Pila()  
p.apilar(10)  
p.apilar(20)  
p.apilar(30)
```

Antes de apilar(30)



Después



↑
cima

Cima y desapilar

```
def cima(self):  
    if self.esta_vacia():  
        raise IndexError("pila vacia")  
    return self._datos[-1]  
  
def desapilar(self):  
    if self.esta_vacia():  
        raise IndexError("pila vacia")  
    return self._datos.pop()
```

Estado inicial: [10, 20, 30].

30	← cima
20	
10	

cima() devuelve 30 y conserva [10, 20, 30].

desapilar() devuelve 30 y deja [10, 20].

pop() sin índice retira el último elemento: el extremo elegido como cima.

Pila enlazada

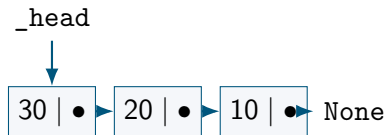
Reutilizamos Nodo(dato, siguiente) del tema de listas.

```
class PilaEnlazada:
    def __init__(self):
        self._head = None
        self._n = 0

    def esta_vacia(self):
        return self._head is None

    def longitud(self):
        return self._n
```

Estado abstracto: (10, 20, 30).



La cabeza es la **cima**.

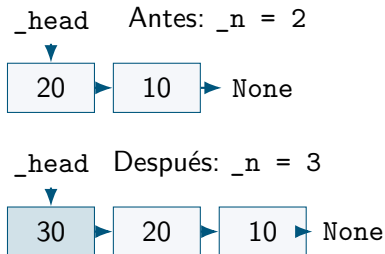
Desde la cabeza recorremos de cima a base: 30, 20, 10.

`_n` cuenta los nodos alcanzables.

Apilar con nodos

Métodos de la clase PilaEnlazada. Ejemplo: `apilar(30)`.

```
def apilar(self, x):  
    self._head = Nodo(x, self._head)  
    self._n += 1  
  
def cima(self):  
    if self.esta_vacia():  
        raise IndexError("pila vacia")  
    return self._head.dato
```



El nuevo nodo enlaza con la cabeza anterior antes de convertirse en la nueva cabeza. `cima()` no cambia enlaces ni contador.

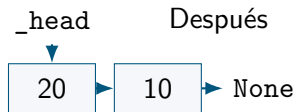
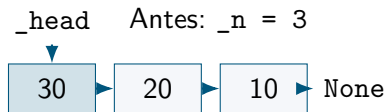
Desapilar con nodos

Método de la clase PilaEnlazada.

```
def desapilar(self):  
    if self.esta_vacia():  
        raise IndexError("pila vacia")  
    dato = self._head.dato  
    self._head = self._head.siguiente  
    self._n -= 1  
    return dato
```

Con un solo nodo, siguiente es None: la nueva cabeza queda vacía y `_n` pasa de 1 a 0.

Desenlazamos el primer nodo; no afirmamos que el objeto se destruya inmediatamente.



Devuelve 30; `_n = 2`

Peor caso, salvo (*): costo amortizado. Sea n el número de elementos.

Operación	Pila con list	PilaEnlazada con contador
Apilar	$\Theta(1)$ (*)	$\Theta(1)$
Desapilar	$\Theta(1)$ (*)	$\Theta(1)$
Consultar cima	$\Theta(1)$	$\Theta(1)$
Consultar longitud	$\Theta(1)$	$\Theta(1)$
Consultar si está vacía	$\Theta(1)$	$\Theta(1)$

(*) En `list` un redimensionamiento puede costar $\Theta(n)$, aunque el promedio por operación en una secuencia es constante.

La ventaja habitual de una lista enlazada es modificar enlaces en una posición conocida. Esto no aporta mucho a una pila, porque el TDA solo permite operar en la cima.

Notación posfija

Definición: notación posfija

Cada operador aparece después de sus operandos. El orden de los símbolos determina la agrupación sin utilizar paréntesis.

Infija	Posfija
$(3 + 4) \cdot 2$	$3\ 4\ +\ 2\ *$

Usaremos una lista de **tokens**: cada elemento es un entero o un símbolo de operador.

```
tokens = [3, 4, "+", 2, "*"]
```

Primero se calcula $3 + 4$; después se multiplica el resultado por 2. No estudiaremos aquí la conversión de infija a posfija.

Precondición: expresión bien formada, con enteros y operadores binarios $+$, $-$, $*$.

Evaluar(tokens)

$P \leftarrow$ pila vacía

Para cada t en tokens:

Si t es un entero:

$P.$ apilar(t)

Si no:

$derecho \leftarrow P.$ desapilar()

$izquierdo \leftarrow P.$ desapilar()

$valor \leftarrow$ Aplicar(t , $izquierdo$, $derecho$)

$P.$ apilar($valor$)

Devolver $P.$ desapilar()

Una expresión bien formada permite retirar dos operandos por operador y deja un único resultado. No añadimos análisis de cadenas ni validación sintáctica.

Orden de operandos

El primero que retiramos es el **derecho**.

Aplicar(t , a , b):

t	resultado
$+$	$a + b$
$-$	$a - b$
$*$	$a \cdot b$

Traza posfija

Entrada: 8 3 - 2 *. Estado de la pila de base a cima.

Token	Acción	Pila
8	Apilar 8.	[8]
3	Apilar 3.	[8, 3]
-	Derecho = 3, izquierdo = 8: apilar $8 - 3 = 5$.	[5]
2	Apilar 2.	[5, 2]
*	Derecho = 2, izquierdo = 5: apilar $5 \cdot 2 = 10$.	[10]

Al terminar, desapilar devuelve 10.

Una pila resulta conveniente cuando necesitamos:

- **Recuperar lo más reciente:** el último elemento agregado sale primero.
- **Manejar anidamiento:** la apertura más reciente se cierra antes.
- **Conservar trabajo pendiente:** una llamada espera el resultado de otra.

El **TDA** determina el comportamiento y la interfaz. La **representación** determina cómo se almacenan los elementos y cuánto cuestan sus operaciones.

`list` con cima al final y una lista enlazada con cima en la cabeza son alternativas para el mismo TDA.